Delicias en Ruta

Tutorial de desarrollo y extensión

Guía programática · rama feature/delicias-en-ruta-safe

Arquitectura · dominio · vistas · tiempo real · persistencia IPFS

Este documento explica cómo modificar la app Delicias en Ruta desde el código: dónde está cada capa, cómo añadir pantallas y casos de uso, cómo cambiar la lógica contable/operativa y cómo funciona la actualización en tiempo real de las vistas.

Índice

- 1. Mapa del código
- 2. Arranque y prueba local
- 3. Flujo de una petición de usuario
- 4. Añadir una pantalla nueva (paso a paso)
- 5. Añadir un caso de uso / cambiar lógica de dominio
- 6. Contabilidad: partida doble y ecuación
- 7. Persistencia IPFS local
- 8. Tiempo real: bus reactivo y vistas vivas
- 9. Roles, permisos y sesión
- 10. Diseño (CSS) sin romper la lógica
- 11. Portable Windows
- 12. Checklist antes de commit
- 13. Errores frecuentes

1. Mapa del código

Todo vive bajo **static/delicias-en-ruta/** (no toca el webapp Svelte de main).

Ruta	Responsabilidad
index.html	Shell HTML: #app + theme.css + main.js
css/theme.css	Diseño Delicias (no lógica)
js/main.js	Boot, splash, login, shell, routing, suscripción reactiva
js/domain/model.js	Estado vacío, plan de cuentas, roles, ecuación
js/domain/engine.js	Casos de uso (partida doble, POS, almacén…)
js/infrastructure/ipfs-store.js	Save/load con CID en localStorage
js/infrastructure/reactive.js	Bus notify/subscribe + multi-pestaña
js/auth/session.js	Sesión en localStorage
js/ui/shell.js	Menú NAV + layout
js/ui/login.js / splash.js	Login y splash
js/ui/views/*.js	Pantallas (forms, dashboard)

El servidor Go (**cmd/abacophy**) solo sirve estáticos y la API de main. Delicias, en modo offline IPFS, **no depende** de la API para su lógica: el cerebro es **DomainEngine**.

2. Arranque y prueba local

```
git checkout feature/delicias-en-ruta-safe
DELICIAS_HOME=1 go run ./cmd/abacophy
# → http://localhost:8090/delicias-en-ruta/

# Solo UI estática (sin API):
cd static/delicias-en-ruta && python3 -m http.server 8088
```

Credenciales offline: admin/admin123 · master/AbacoPhy#Master1 · caja/caja123.

3. Flujo de una petición de usuario

```
Usuario hace clic en "Guardar ingreso"
        ■
        ▼
Vista (forms.js) llama eng.createIncomeEntry({…})
        ■
        ▼
DomainEngine valida → _createEntry → applyDoubleEntry
        ■
        ▼
persist() → IpfsBusinessStore.save (CID) → notify('state')
        ■
        ▼
subscribe en main.js → paintApp() repinta la vista activa
        ■
        ▼
Otras pestañas reciben storage/BroadcastChannel y también repintan
```

4. Añadir una pantalla nueva (paso a paso)

4.1 Registrar en el menú

Archivo: **js/ui/shell.js** — array NAV

```
{ group: 'Operación', items: [
  …
  { id: 'mi-modulo', label: 'Mi módulo' },
]},
```

4.2 Implementar la vista

Archivo: **js/ui/views/forms.js** (o un archivo nuevo importado desde main.js)

```
export function renderMiModulo(el, eng, redraw) {
  el.innerHTML = `
    <div class="card form-card">
      <h3>Mi módulo</h3>
      <button type="button" class="btn" id="ok">Acción</button>
    </div>`;
  el.querySelector('#ok').onclick = async () => {
    await eng.miCasoDeUso({ /* datos */ });
    redraw(); // o confiar en notify() tras persist
  };
}
```

4.3 Enrutar en main.js

```
case 'mi-modulo': V.renderMiModulo(el, eng, redraw); break;
```

Añade el id a **LIVE_SCREENS** si debe refrescarse en tiempo real.

4.4 Permisos

En **model.js** (rolePermissions por defecto) y en la pantalla Permisos UI. Roles nuevos: nomenclador Roles + createRole en el engine.

5. Añadir un caso de uso / cambiar lógica

Archivo principal: **js/domain/engine.js**.

```
async miCasoDeUso({ campo }) {
  if (!campo) throw new Error('Campo obligatorio');
  // mutar this.state …
  this.audit('mi.evento', campo);
  return this.persist(); // ← dispara notify → UI al día
}
```

- Validaciones de negocio van en el engine (como los use cases de main), no en el HTML.
- Tras mutar estado, siempre **persist()** para IPFS + tiempo real.
- Estado inicial / seeds: **model.js** → emptyState(), defaultAccounts(), defaultRoles().
- No edites main ni webapp/ Svelte desde esta rama salvo merge explícito.

6. Contabilidad: partida doble y ecuación

Espejo de **internal/domain/ledger.go** (ApplyDoubleEntry + EquationSnapshot):

- Ingreso: Debe activo (Caja) | Haber cuenta de ingreso.
- Gasto: Debe gasto | Haber activo.
- Ecuación: Activo = Pasivo + Patrimonio + (Ingresos – Gastos).

Métodos clave: applyDoubleEntry, _postJournal, equationSnapshot, getTAccount, getSituacionFinanciera, getEstadoResultados, getEcuacionAmpliada.

Si cambias la lógica de asientos, actualiza también las pestañas de Reportes (ecuación, cuentas T, diario) — ya leen del engine.

7. Persistencia IPFS local

Archivo: **js/infrastructure/ipfs-store.js**.

```
save(state) → { cid, prevCid, ts, state } en localStorage clave delicias.ipfs.v2
history() → cadena de CIDs (pantalla Traza / Salvas)
exportSnapshot / importSnapshot → backup JSON
```

Cada persist genera un nuevo CID (hash SHA-256). Las Salvas guardan snapshots restaurables. Para depurar: Application → Local Storage en DevTools del navegador.

8. Tiempo real: bus reactivo y vistas vivas

Archivo: **js/infrastructure/reactive.js** + suscripción en **main.js**.

Cómo funciona

- **notify('state')** se llama al final de DomainEngine.persist().
- **subscribe(fn)** en main.js vuelve a ejecutar paintApp() si la pantalla está en LIVE_SCREENS.
- **BroadcastChannel** + evento **storage**: otras pestañas del mismo origen recargan estado y repintan.

Activar tiempo real en una vista nueva

```
// main.js
const LIVE_SCREENS = new Set([
  'dashboard', 'ingresos', /* … */, 'mi-modulo',
]);
```

No hace falta llamar redraw() a mano si el caso de uso termina en persist(); redraw() sigue siendo útil para feedback inmediato en formularios complejos (p. ej. POS).

Probar multi-pestaña

- Abre dos ventanas en /delicias-en-ruta/ con el mismo origen.
- En una registra un ingreso; en la otra, Dashboard/Reportes deben actualizarse solos.

9. Roles, permisos y sesión

- Sesión: js/auth/session.js (localStorage delicias.session.v1).
- Login: eng.login → setSession → mountShell.
- canAccess(role, screenId) en el engine; master siempre pasa.
- CRUD roles: eng.createRole / updateRole / deleteRole + pantalla Roles.
- Permisos UI: rolePermissions[role][screenId] = true/false.

10. Diseño (CSS) sin romper la lógica

Solo **css/theme.css** y clases usadas en las vistas. Clases útiles: card, form-card, btn, btn-ghost, kpi, grid, table-wrap, tabs-bar, t-account, equation-card.

Logo fijo: .brand-fixed + .brand-logo. No muevas assets/logo.png sin actualizar rutas relativas.

11. Portable Windows

```
./scripts/build-delicias-windows.sh
# → dist/delicias-windows/DeliciasEnRuta.exe
#    Iniciar-Delicias-en-Ruta.bat
```

Copia la carpeta completa. DELICIAS_HOME=1 redirige / a /delicias-en-ruta/.

12. Checklist antes de commit

- Cambios solo bajo static/delicias-en-ruta/ (y docs/scripts de esta rama).
- Nuevo módulo: NAV + vista + case en main.js + LIVE_SCREENS + permisos.
- Nueva lógica: método en engine + validaciones + persist() + audit().
- Probar login, un ingreso, ecuación en Reportes, y dos pestañas (tiempo real).
- No hacer merge a main hasta validar con el negocio.

13. Errores frecuentes

- **Sigo viendo 5 cuentas:** localStorage antiguo — ventana privada o borrar clave delicias.ipfs.v2.
- **La vista no se actualiza:** ¿el método llama persist()? ¿el id está en LIVE_SCREENS?
- **Acceso denegado:** rolePermissions del rol no incluye esa pantalla.
- **Ecuación desbalanceada:** asientos a mano sin contrapartida; usar formularios de ingreso/gasto con cuenta + activo.
- **Rompi main:** no edites webapp/ ni merges sin revisión; Delicias es rama dedicada.